

When AI Exposes Your Guardrails

Moshe Zadka – <https://cobordism.com>

In order to convert this to PDF, use ides:

```
pip install ides
python -m ides
```

Two Narratives

“Quality tanked”

“Productivity tripled”

Both right.

There are two loud camps in the AI-assisted coding debate. One says quality cratered. The other says productivity exploded. Both have data. Both are vocal. My claim is that they are both right, and they are experiencing the same underlying effect from opposite sides.

Guardrails Compound

Catch mistakes

Scale cheaply

Or: entropy

Guardrails, linters, tests, CI checks, catch recurring mistakes before a human ever sees them. They are gentler than review comments, cheaper than context switches, and they scale. Teams that compound guardrails over years accumulate cognitive capacity. Teams that do not watch entropy grow: humans get tired, violations become precedent, inconsistency wins.

AI Changes the Velocity

3–4x, accelerating

Gap widens

Guardrails: load-bearing

AI has dramatically increased developer productivity: 3 to 4 times, and climbing with each model release. That is the real issue. Weak guardrails plus low velocity was barely functional. Weak guardrails plus AI velocity is a quality crisis. Strong guardrails plus AI velocity just ships more features. The velocity change is what makes process quality load-bearing instead of optional.

The Organizational Trap

Easy: blame AI

Correct: fix process

Same maturity prevents & repairs

Blaming the AI is politically costless. Nobody has to admit past practices were inadequate. Fixing the processes requires cultural change and infrastructure investment. Here is the trap: the organizational maturity needed to respond correctly is the same maturity that would have prevented the problem. The teams struggling most are, by definition, the teams least equipped to fix it, and the gap is widening at an accelerating rate.

What To Do

Problems pre-existed

Audit; automate

Correctness by default

If AI exposed quality problems, you had quality problems. AI just raised velocity until they became visible. The concrete action: audit what your review process currently catches, and invest in automation proportional to the velocity gains you are seeing. Make correctness the path of least resistance, not a tax paid on every change.

This Week

Automate one review comment.

So here is what I am asking you to do. This week, before your next AI-assisted pull request: pick one comment you find yourself leaving in code reviews over and over, and turn it into a linter rule, a test, or a CI check. One guardrail, shipped this week. That is how you start compounding. Find me at cobordism.com and tell me what you built.